UNIVERSIDADE ESTADUAL DA PARAÍBA

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Projeto Integrador e Prática Profissional II — 2026.2

ENTREGA 2 — MODELAGEM UML

Casos de uso, classes e comportamento do sistema

ADS 7 — 23 de outubro · ADS 8 — 26 de outubro de 2026

Campo	Preencher
Nome da equipe	NoBugs
Turma	(X) Virtual ADS 7 ( ) Virtual ADS 8
Nome do sistema	Cidade Ouve
Link do repositório	https://github.com/KauaFerreira-dev-eng/NoBugs.git
Ferramenta de modelagem usada	Plant UML
Data de entrega	23/10/2026

1. Identificação da equipe

Nome completo	Usuário no GitHub	Papel principal nesta etapa
Janaina Gabrielly Barbosa	@janaina-exe	revisou a modelagem contra os requisitos
José Luann de Lira Fidelis	@luannlira	desenhou o diagrama de classes
Karla Geovanna Gomes Trajano	@Geovanatrajano	escreveu o dicionário de classes e a descrição dos cenários
Kauã Ferreira	@KauaFerreira-dev-eng	desenhou os diagramas de sequência

2. O que mudou desde o Checkpoint 1

O que estava assim	Virou	Por quê
RF01 - O sistema deve gerenciar ocorrências	RF01 e RF02, separados: cadastrar ocorrência e consultar ocorrências	'Gerenciar' escondia duas operações diferentes, com atores e telas diferentes

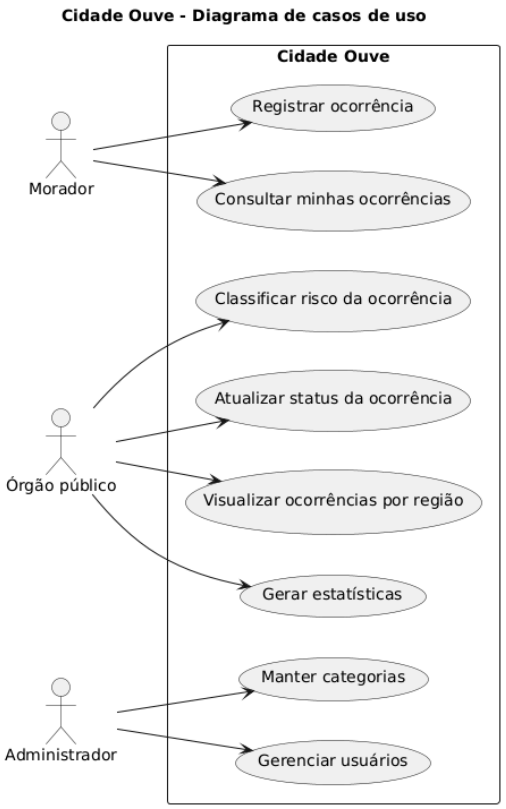
O que estava assim	Virou	Por quê
RF03 - O sistema deve permitir acompanhar o problema	RF03: o cidadão consulta o status de ocorrência (aberta, em análise, resolvida), e RF04: o órgão público atualiza o status	“Acompanhar” era vago. Não dizia quem consulta, quem atualiza e quais são os estados possíveis
RF06 - O sistema deve gerar estatísticas	RF06: gerar relatório de ocorrências por tipo (buraco, lixo, iluminação) e RF07: gerar relatório por bairro/período	“Estatísticas” não dizia quais dados nem como seriam filtrados, então não dava para testar
RF02 — O cidadão deve registrar a ocorrência com foto	Removida a foto: o cadastro passa a ter só tipo do problema, descrição e localização	Enviar foto exigia armazenamento de arquivos e deixava o sistema mais complexo. A equipe tem pouca experiência em Java e o escopo precisava ser viável

3. Atores do sistema

Ator	Papel: o que ele quer do sistema
Morador	Registrar um problema urbano e acompanhar se ele é resolvido
Órgão público	Atualizar o status das ocorrências sob sua responsabilidade e consultar estatísticas para priorizar onde agir.
Administrador	Manter as categorias de problema atualizadas e gerenciar usuários.

4. Diagrama de casos de uso

Campo	Preencher
Arquivo editável no repositório	/docs/puml/casos-de-uso1.puml
Quantidade de casos de uso	8



5. Especificação dos casos de uso

Campo	Preencher
Caso de uso	Registrar ocorrência
Ator principal	Morador
Pré-condição	O morador acessa o sistema e possui as informações necessárias para registrar o problema.
Pós-condição	A ocorrência é registrada com protocolo e status inicial.
Fluxo principal	1. O morador solicita o registro de uma ocorrência. 2. O sistema apresenta os campos para preenchimento. 3. O morador informa a descrição, a categoria e a localização do problema. 4. O sistema valida as informações. 5. O sistema registra a ocorrência e gera seu protocolo. 6. O sistema confirma o registro ao morador.
Fluxo alternativo	3a. O morador deixa de preencher um campo obrigatório. 3b. O sistema informa que os campos obrigatórios precisam ser preenchidos e solicita a correção. 3c. Após a correção, o fluxo principal continua.
Regras de negócio aplicadas	RN01 e RN02.

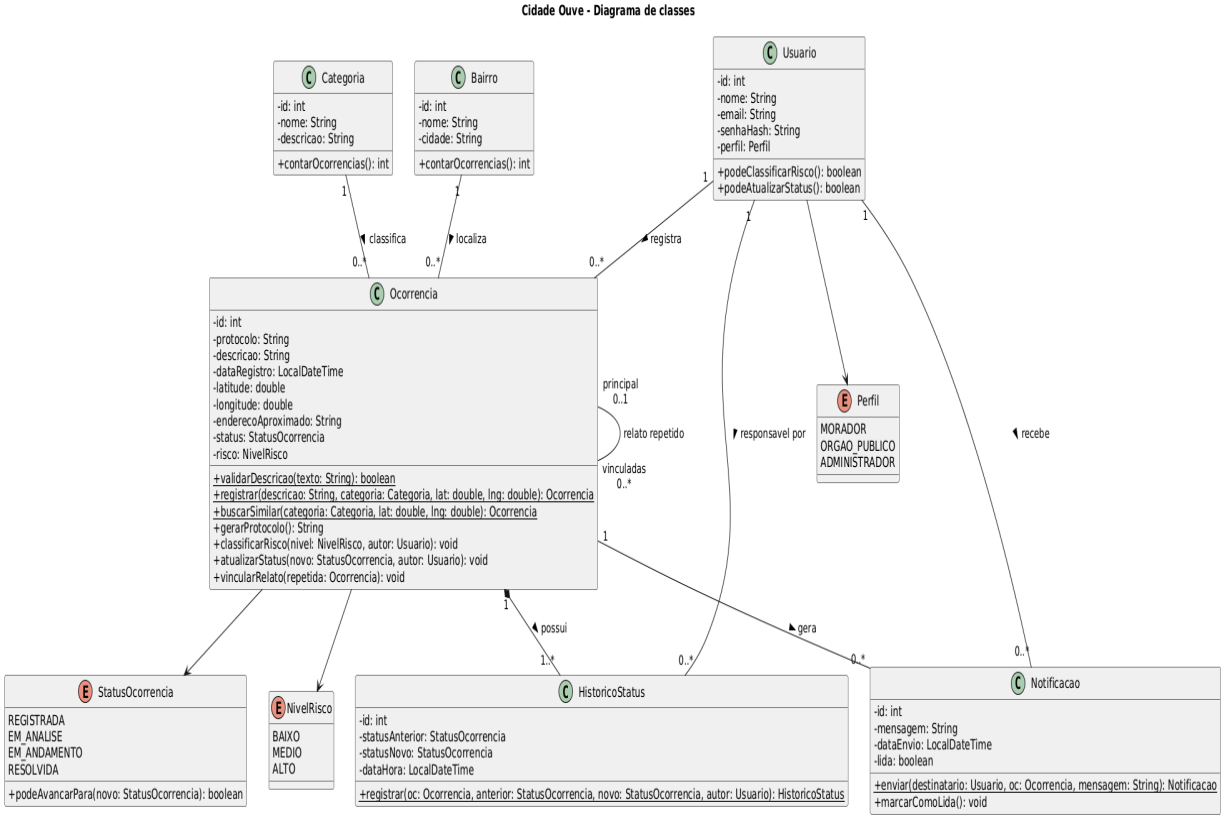
Campo	Preencher
Caso de uso	Registrar ocorrência
Ator principal	Morador
Pré-condição	O morador acessa o sistema e possui as informações necessárias para registrar o problema.
Pós-condição	A ocorrência é registrada com protocolo e status inicial.
Fluxo principal	1. O morador solicita o registro de uma ocorrência. 2. O sistema apresenta os campos para preenchimento. 3. O morador informa a descrição, a categoria e a localização do problema. 4. O sistema valida as informações. 5. O sistema registra a ocorrência e gera seu protocolo. 6. O sistema confirma o registro ao morador.
Fluxo alternativo	3a. O morador deixa de preencher um campo obrigatório. 3b. O sistema informa que os campos obrigatórios precisam ser preenchidos e solicita a correção. 3c. Após a correção, o fluxo principal continua.
Regras de negócio aplicadas	RN01 e RN02.

6. Rastreabilidade

Requisito	Caso de uso	Ator	Classes envolvidas
RF01, RF02, RF03	Registrar ocorrência	Morador	Usuario, Ocorrencia, Categoria, Bairro, HistoricoStatus
RF04	Classificar risco	Órgão público	Usuario, Ocorrencia
RF07	Visualizar ocorrências por região	Órgão público	Ocorrencia, Bairro, Categoria
RF08	Gerar estatísticas	Órgão público	Ocorrencia, Bairro, Categoria
RF05, RF09	Registrar ocorrência	Morador	Ocorrencia, Categoria
RF06	Consultar minhas ocorrências	Morador	Usuario, Ocorrencia, HistoricoStatus

7. Diagrama de classes

Campo	Preencher
Arquivo editável no repositório	/puml/diagrama-de-classes.puml
Quantidade de classes	6 classes (Categoria, Bairro, Usuario, Ocorrencia, HistoricoStatus, Notificacao).



8. Dicionário de classes

Classe	O que representa no mundo real	Atributos, com tipo	Métodos
Usuário	Pessoa que usa o sistema	*i*d: int; nome;String; email: Strings; senhaHash; String; perfil: Perfil	podeClassificarRisco() : boolean; podeAtualizarStatus(): boolean
Categoria	Tipo de problema urbano reportável	id: int; nome: String; cidade:String.	contarOcorrencias(): int
Bairro	Região onde a ocorrência acontece	id: int; nome: String; cidade: String	contarOcorrencias(): int
Ocorrencia	Problema relatado, do registro à resolução	id: int; protocolo: String; descricao: String; dataRegistro: LocalDateTime; latitude: double; enderecoAproximado: String; status: StatusOcorrencia; risco: NivelRisco.	registrar(); buscarSimilar(); gerarProtocolo(); classificarRisco(); atualizarStatus(); vincularRelato()

Classe	O que representa no mundo real	Atributos, com tipo	Métodos
HistoricoStatus	O registro de cada mudança de status pela qual uma ocorrência já passou, e quem a fez	id: int; statusAnterior: StatusOcorrencia; statusNovo: StatusOcorrencia; dataHora: LocalDateTime	registrar()
Notificacao	Aviso enviado ao morador	id: int; mensagem: String; dataEnvio: LocalDateTime; lida: boolean	enviar(); marcarComoLida()
Ocorrencia	Problema relatado, do registro à resolução	id: int; protocolo: String; descricao: String; dataRegistro: LocalDateTime; latitude: double; longitude: double; enderecoAproximado: String; status: StatusOcorrencia; risco: NivelRisco.	validarDescricao(); registrar(); buscarSimilar(); gerarProtocolo(); classificarRisco(); atualizarStatus(); vincularRelato()

9. Relacionamentos e multiplicidade

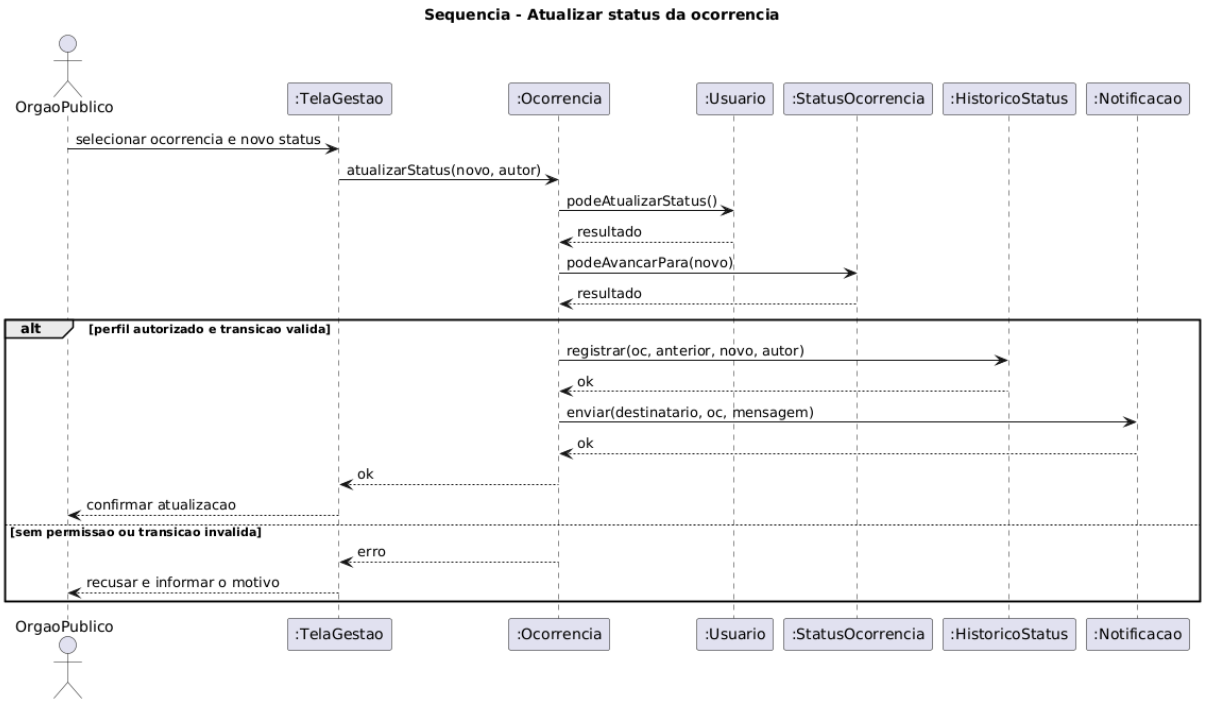
Classe A	Mult.	Classe B	Mult.	Como se lê
Categoria 1	1	Ocorrencia	0..*	Uma categoria pode estar associada a zero ou muitas ocorrências; cada ocorrência possui uma categoria.
Bairro 1	1	Ocorrencia	0..*	Um bairro pode ter zero ou muitas ocorrências; cada ocorrência pertence a um bairro.
Usuario 1	1	Ocorrencia	0..*	Um usuário pode registrar zero ou muitas ocorrências; cada ocorrência está associada a um usuário.
Ocorrencia 1	1	HistoricoStatus	0..*	Uma ocorrência pode ter vários registros de histórico; cada registro pertence a uma ocorrência.
Usuario 1	1	HistoricoStatus	0..*	Um usuário pode realizar várias mudanças de status; cada registro de histórico identifica o autor da mudança.
Usuario 1	1	Notificacao	0..*	Um usuário pode receber várias notificações; cada notificação é destinada a um usuário.

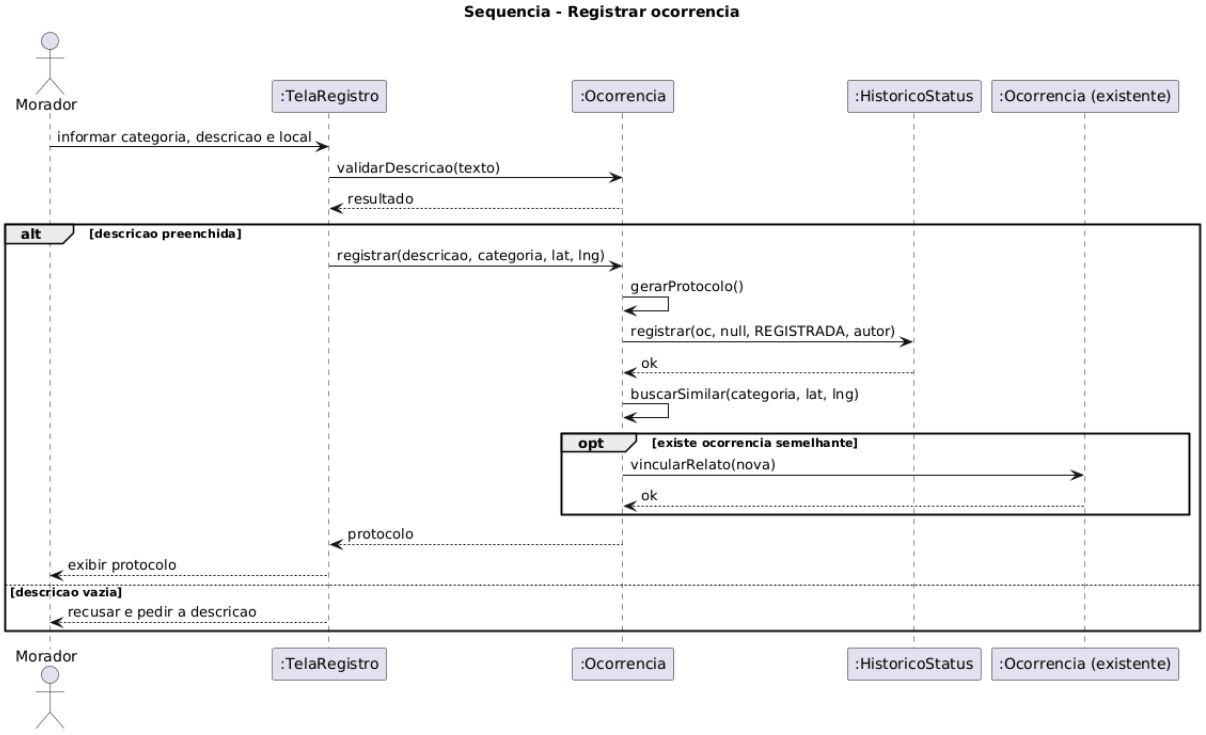
10. Onde mora cada regra de negócio

Regra	Enunciado	Classe	Método que a implementa
RN01	A ocorrência só é registrada com descrição, categoria e localização preenchidas	Ocorrência	registrar()
RN02	Toda ocorrência nova começa com o status REGISTRADA	Ocorrência	registrar() e o enum StatusOcorrencia
RN03	O status só avança na ordem REGISTRADA → EM ANÁLISE → EM ANDAMENTO → RESOLVIDA, sem pular nem voltar	StatusOcorrencia	podeAvancarPara(novo), usado em Ocorrencia.atualizarStatus()
RN04	Só quem tem perfil autorizado (órgão público) atualiza o status	Usuario	podeAtualizarStatus()
RN05	Toda mudança de status gera um registro com status anterior, status novo, autor e data/hora	HistoricoStatus	registrar()

11. Diagramas de sequência

Campo	Diagrama 1	Diagrama 2
Caso de uso detalhado	Registrar ocorrência	Atualizar status da ocorrência
Arquivo editável no repositório	/puml/sequencia-registrar-ocorrencia.puml	/puml/sequencia-atualizar-status.puml
Classes que participam	Ocorrencia, HistoricoStatus	Ocorrencia, Usuario, StatusOcorrencia, HistoricoStatus, Notificacao
Condição do bloco alternativo	Descrição preenchida	Perfil autorizado e transição de status válida
O que acontece em cada caminho	**Preenchida:** o sistema cria a ocorrência, gera o protocolo, grava o primeiro histórico (REGISTRADA) e exibe o protocolo. Se já existe ocorrência semelhante, o relato é vinculado a ela. **Vazia:** o sistema recusa e pede a descrição.	**Válida:** o sistema grava o histórico, notifica o morador e confirma ao órgão público. **Inválida ou sem permissão:** o sistema recusa e informa o motivo.





12. Decisões de modelagem

Decisão ou descarte	Alternativa considerada	Por que decidimos assim
Categoria e Bairro viraram classes	Enums	O administrador cadastra categorias e as estatísticas agrupam por eles

Decisão ou descarte	Alternativa considerada	Por que decidimos assim
HistoricoStatus virou classe (composição)	Só o campo status	Cada mudança guarda quem, quando e de qual status para qual
Perfil virou atributo e enum	Herança (Morador, Orgao, Admin)	Muda só a permissão, não os dados
Relato repetido virou autoassociação	Classe Relato	Modelo mais enxuto; vira classe se for preciso guardar quem relatou
Estatística e painel não viraram classe	Classe Estatistica	São saída de consulta

13. O que faremos até a Entrega 3

Semana	O que vamos fazer	Quem é responsável
26/10 a 30/10	Converter o diagrama de classes em modelo entidade-relacionamento e decidir o que fica fixo no código (enums) e o que vira tabela	Kauã e Janaina
02/11 a 06/11	Definir chaves primárias e estrangeiras, tipos das colunas e normalização	Karla e Kauã
09/11 a 13/11	Escrever os scripts SQL de criação das tabelas e carga inicial (categorias, bairros)	Janaina e Luann
16/11 a 20/11	Mapear as classes como entidades no Spring Boot e ligar ao banco	Luann e Karla
23/11 a 27/11	Revisar o modelo de dados e atualizar a documentação e os diagramas com o que mudou	Todos

14. Declaração de uso de inteligência artificial

Quem usou	Ferramenta	Em que parte	Para quê
Kauã	Claude	Seção 7	Ajuda na criação do diagrama de classes
Luann	Claude	Seção 8	Ajuda na associação das idéias em conjunto com a seção 7 e correção.

Anexo — Confiram antes de entregar

	Item
(x)	Todas as seções preenchidas, sem linhas de instrução (▸) e sem as linhas de exemplo em cinza
(x)	Os três diagramas inseridos como imagem legível e com arquivo editável no repositório
(x)	Todo requisito essencial aparece na tabela de rastreabilidade
(x)	Nenhum caso de uso é passo interno — nada de 'Validar CPF' ou 'Abrir tela'
(x)	Todo caso de uso essencial tem fluxo alternativo
(x)	Toda associação do diagrama de classes tem multiplicidade nas duas pontas
(x)	Nenhuma classe é tela, e nenhuma classe tem nome de verbo
(x)	Toda regra de negócio tem classe e método onde morar
(x)	Toda mensagem dos diagramas de sequência existe como método na seção 8
(x)	Cada diagrama de sequência tem um bloco alternativo
(x)	Declaração de uso de IA preenchida
(x)	Arquivo em PDF, em /docs, com o nome entrega-2-modelagem.pdf